

PARTICIPANT STUDY GUIDE · DAY 2

Reporting, Dashboards & Administration

Splunk Fundamentals · Day 2 — Participant Study Guide

- Module 6 — Statistical Analysis Using SPL
- Module 7 — Lookups and Data Enrichment
- Module 8 — Reports and Visualisations
- Module 9 — Dashboards
- Module 10 — Alerts and Monitoring
- Module 11 — Basic Splunk Administration
- Case studies, knowledge checks, quick reference

Vivek Arora

Corporate Trainer, Author & Consultant
StackMaster Tech, Bengaluru

Running scenario: **NovaMart** (fictional e-commerce retailer)
Applies to Splunk Enterprise 9.x / 10.x & Splunk Cloud Platform
Edition 1.0 · September 2026

Contents

About Day 2	3
NovaMart, the Morning After	5
Statistical Analysis Using SPL	6
6.1 Three Commands, One Idea	6
6.2 Aggregation Functions	6
6.3 Grouping Results with by	7
6.4 Working with Time-Based Data	8
6.5 top, rare and where	9
6.6 Combining eval with Statistical Commands	10
Lookups and Data Enrichment	13
7.1 Why Lookups?	13
7.2 Creating CSV Lookups and Lookup Definitions	13
7.3 Manual and Automatic Lookups	15
7.4 lookup, inputlookup and outputlookup	15
7.5 Enriching Events with External Information	16
Reports and Visualisations	18
8.1 Saving Searches as Reports	18
8.2 Creating Tables and Charts	18
8.3 Selecting and Configuring Visualisations	19
8.4 Scheduling, Exporting and Sharing Reports	20
8.5 Managing Permissions and Ownership	21
Dashboards	23
9.1 Dashboard Frameworks	23
9.2 Creating Dashboards and Search-Based Panels	23
9.3 Tables, Charts and Single-Value Panels	24
9.4 Inputs, Tokens and Time-Range Controls	25
9.5 Interactivity, Arrangement and Sharing	26
Alerts and Monitoring	29
10.1 How an Alert Works	29
10.2 Trigger Conditions, Throttling and Alert Actions	29
10.3 Email and Webhook Overview	31
10.4 Alert Management, Troubleshooting and Best Practice	31
Basic Splunk Administration	35
11.1 Configuration Directory Structure	35
11.2 inputs.conf, outputs.conf, props.conf and transforms.conf	36
11.3 Index and Retention Concepts	37
11.4 User Roles, Capabilities and Search Permissions	38
11.5 Monitoring Console Overview	39
11.6 Troubleshooting Using _internal Logs	39
Day 2 Summary	42
Day 2 Quick Reference	43

Glossary	43
Appendix A: Knowledge Check Answers	45

About Day 2

On Day 1 you brought NovaMart's machine data into Splunk and learned to search it. You found a checkout outage and a brute-force attack, but only because you went looking. **Day 2 is about making Splunk work for you:** turning searches into statistics, enriching them with business context, publishing reports and dashboards, raising alerts automatically, and running the platform itself.

Analyse

`stats`, `chart`, `timechart`, `top`, `rare`, `where` and `eval` turn thousands of events into answers.

Share

Lookups add business meaning; reports and dashboards put the answers in front of the right people.

Operate

Alerts watch the data for you. Administration keeps the platform healthy, secure and correctly configured.

Day 2 Agenda

Time	Session	Hands-on
09:30 – 10:45	Module 6 — Statistical Analysis Using SPL	Lab 8: statistical searches with stats and timechart Lab 9: application performance and error trends
10:45 – 11:00	Break	
11:00 – 11:45	Module 7 — Lookups and Data Enrichment	Lab 10: enrich data with CSV lookups
11:45 – 12:45	Module 8 — Reports and Visualisations	Lab 11: create and schedule a report
12:45 – 13:30	Lunch	
13:30 – 14:45	Module 9 — Dashboards	Lab 12: build an interactive operational dashboard
14:45 – 15:45	Module 10 — Alerts and Monitoring	Lab 13: configure a threshold-based alert
15:45 – 16:00	Break	
16:00 – 17:30	Module 11 — Basic Splunk Administration	Lab 14: troubleshoot a data-ingestion issue Lab 15: administration essentials

Learning Outcomes

By the end of Day 2 you will be able to:

1. Summarise data with `stats`, `chart` and `timechart`, using `count`, `sum`, `avg`, `min`, `max` and `dc`, grouped with `by`.
2. Analyse time-based data, filter results with `where`, and combine `eval` with statistical commands to calculate rates and KPIs.
3. Create CSV lookups, lookup definitions and automatic lookups, and enrich events with `lookup` and `inputlookup`.
4. Save searches as reports, choose and configure visualisations, and schedule, export and share reports with the right permissions.

5. Build dashboards with tables, charts and single values, driven by inputs, tokens and time-range controls, with basic drilldown.
6. Create scheduled and real-time alerts with trigger conditions, throttling, and email or webhook actions, and troubleshoot them.
7. Navigate Splunk's configuration directories; read `inputs.conf`, `outputs.conf`, `props.conf` and `transforms.conf`; manage indexes, retention, roles and capabilities; use the Monitoring Console and `_internal` logs to troubleshoot.

NovaMart, the Morning After

Your Day 1 investigation reached the leadership team. The CTO's response was blunt: *"We lost ₹2 lakh of orders and a hacker created an account on our bastion host, and we only found out because someone went looking. That can't happen again."* The Platform Engineering team has been given five goals, and they map directly to today's modules.

Stakeholder asks for...	Delivered with...	Module / Lab
SRE: "What's our checkout error rate and payment latency, per day and per hour?"	<code>stats</code> , <code>timechart</code> , <code>eval</code> + <code>where</code>	Module 6 · Labs 8–9
Business: "Revenue by category and region, against target, with product names"	CSV lookups, automatic lookups	Module 7 · Lab 10
Leadership: "A revenue report in my inbox every morning"	Scheduled report	Module 8 · Lab 11
NOC: "One screen that shows platform health"	Interactive dashboard	Module 9 · Lab 12
On-call and security: "Tell us the moment checkout breaks or someone brute-forces SSH"	Threshold alerts with throttling and webhooks	Module 10 · Lab 13
Platform: "Onboard the new warehouse feed, lock down who sees security data, set retention"	Configuration files, indexes, roles, the Monitoring Console	Module 11 · Labs 14–15

Your data for today

Day 2 uses the **same NovaMart data** as Day 1: `index=novamart`, 14–20 September 2026. The dataset (v1.1) and the `novamart_lab` Splunk app (v1.1) add four things:

New item	Used for
<code>lookups/network_zones.csv</code> , <code>region_targets.csv</code> , <code>gateway_sla.csv</code> , <code>known_bots.csv</code>	Module 7 and Lab 10: CIDR matching, targets, SLAs, bot filtering
<code>live/novamart_live.py</code>	Writes events with the current time (web03, app01, bastion01), so dashboards update and alerts can fire. Modes: <code>normal</code> , <code>incident</code> , <code>bruteforce</code> .
<code>tools/webhook_listener.py</code>	A local receiver that prints the webhook payloads your alerts send
<code>troubleshoot/inventory_wh.log</code> + <code>inputs_broken.conf</code>	A new warehouse feed with deliberate onboarding mistakes, for Lab 14

IMPORTANT · TIME

Historical searches use **All time** or explicit dates between 14 and 20 September 2026. Live data (Labs 12–13) uses relative times such as *Last 15 minutes*. Keep your user time zone set to **Asia/Kolkata** so that the clock times match this guide. Counts quoted for historical data exclude the live data; if the live generator is running, restrict the time range to 14–20 September 2026 (e.g. *Date Range: 14/09/2026 – 20/09/2026*) to reproduce them exactly.

Statistical Analysis Using SPL

stats · chart · timechart · count, sum, avg, min, max, dc · by · time-based data · top, rare, where · eval with statistics

AFTER THIS MODULE YOU CAN

- choose between `stats`, `chart` and `timechart` for a given question
- use the core aggregation functions and group results with `by`
- analyse data over time with `timechart`, `span`, `bin` and `strftime`
- rank values with `top` and `rare`, and filter results with `where`
- combine `eval` with statistical commands to build KPIs such as error rate and conversion

6.1 Three Commands, One Idea

All three commands **aggregate** events into a statistics table. They differ in the *shape* of the table they produce, which determines how it can be charted.

stats count by host, status			chart count over host by status					timechart span=1d count by host		
host	status	count	host	200	404	500	503 ...	_time	web01	web02
web01	200	10573	web01	10573	192	23	20	2026-09-14	1027	1169
web01	404	192	web02	11683	48	15	15	2026-09-15	995	1221
web01	503	20						...		
web02	200	11683						2026-09-18	4177	4480
...	...	(12 rows)						...	(7 rows)	
"Long" table: one row per combination. Best for tables and further processing.			"Wide" table: row field × column field. Best for column/bar charts.					Rows are always time buckets (_time). Best for line/area charts over time.		

Figure 6.1 — The same web data summarised by stats, chart and timechart

Command	Syntax (simplified)	Use it when you want...
<code>stats</code>	<code>stats <func>(field) [AS name] ... [by f1, f2 ...]</code>	A table of totals per group, any number of group-by fields; the most flexible
<code>chart</code>	<code>chart <func>(field) over <row> by <column></code>	A two-dimensional matrix to compare categories side by side
<code>timechart</code>	<code>timechart [span=<t>] <func>(field) [by <split>]</code>	Trends over time; <code>_time</code> is always the x-axis

6.2 Aggregation Functions

Function	Returns	NovaMart example
<code>count</code>	Number of events (or rows)	<code>stats count</code>
<code>count(field)</code>	Number of events in which <i>field</i> exists	<code>stats count(error_code) = 80</code>
<code>dc(field)</code>	Distinct count: number of unique values	<code>stats dc(clientip) = 425 visitors</code>
<code>sum(field)</code>	Total of a numeric field	<code>stats sum(amount) = revenue</code>

Function	Returns	NovaMart example
<code>avg(field)</code> / <code>mean</code>	Arithmetic mean	<code>stats avg(latency_ms)</code>
<code>min(field)</code> / <code>max(field)</code>	Smallest / largest value	<code>stats max(response_ms)</code>
<code>median</code> , <code>perc95(field)</code>	50th / 95th percentile (approximate on large sets)	<code>stats perc95(latency_ms)</code>
<code>range</code> , <code>stdev</code> , <code>var</code>	Spread of the values	<code>stats stdev(amount)</code>
<code>values(field)</code> / <code>list(field)</code>	Unique values / all values (multivalued)	<code>stats values(user) by src_ip</code>
<code>earliest</code> , <code>latest</code>	Value from the oldest / newest event	<code>stats latest(status) by host</code>
<code>min(_time)</code> , <code>max(_time)</code>	First and last time seen (epoch; format with <code>strftime</code>)	<code>stats min(_time) AS first_seen by clientip</code>

EXAMPLE · SEVERAL FUNCTIONS AT ONCE, WITH AS

```
index=novamart sourcetype=access_combined
| stats count AS requests, dc(clientip) AS visitors, sum(bytes) AS bytes_sent,
    avg(bytes) AS avg_bytes, min(bytes) AS min_bytes, max(bytes) AS max_bytes by host
| eval avg_bytes=round(avg_bytes, 1)
```

host	requests	visitors	bytes_sent	avg_bytes	min_bytes	max_bytes
web01	10856	414	52683824	4853.0	17	9500
web02	11785	417	58424723	4957.5	17	9500

Almost every visitor used both servers, because the load balancer spreads sessions across them. That is why 414 + 417 is much more than the 425 distinct IPs overall.

WATCH OUT · FIELDS DISAPPEAR AFTER STATS

`stats` keeps **only** the aggregated fields and the `by` fields. After `| stats count by host` there is no `_raw`, `status` or `_time` any more, so a following `| where status ≥ 500` matches nothing. Filter *before* `stats`, or include the field in `by`.

6.3 Grouping Results with by

`by` splits the calculation into groups. Several `by` fields create one row per *combination* of values that actually occurs.

```
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS
| stats count AS orders, sum(amount) AS revenue, avg(amount) AS aov, dc(customer_id) AS customers by region
| eval aov=round(aov, 0)
| sort - revenue
```

region	orders	revenue	aov	customers
North	254	1770840	6972	143

region	orders	revenue	aov	customers
South	294	1717583	5842	160
West	164	996654	6077	99
East	100	707421	7074	60

South has the most orders, but North earns more, because its average order value (AOV) is higher. East has the fewest orders but the highest AOV. Questions like these are why the business team loves `stats ... by`.

chart: over and by

```
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS
| chart sum(amount) over region by payment_method
```

region	CARD	COD	NETBANKING	UPI	WALLET
East	284799	42884	91935	255709	32094
North	561543	169767	203022	702275	134233
South	782604	156069	270005	438876	70029
West	342682	184426	130083	309378	30085

- `over` sets the rows (x-axis); `by` sets the columns (series). `chart ... by region, payment_method` is an equivalent shorthand.
- By default, only the top 10 column values are shown and the rest are grouped as `OTHER`. Change this with `limit=0` (show all) or `useother=false`.

6.4 Working with Time-Based Data

timechart

```
index=novamart sourcetype=access_combined | timechart span=1d count by host
```

_time	web01	web02
2026-09-14	1027	1169
2026-09-15	995	1221
2026-09-16	1244	1089
2026-09-17	1065	1132
2026-09-18	4177	4480
2026-09-19	1162	1339
2026-09-20	1186	1355

- `span=` sets the bucket size: `5m`, `1h`, `1d`, `1w` ... Without it, Splunk picks a span that fits the time range.
- Buckets with no events show `0` for `count`, so the timeline has no gaps. Add `cont=false` to drop empty buckets.

- Like `chart`, `timechart ... by` shows the top 10 series plus `OTHER` (`limit=`, `useother=`).
- Day buckets start at midnight in **your** time zone, which is another reason to set Asia/Kolkata.

Other ways to work with time

Technique	Example and result
<code>bin _time span=1h + stats</code>	<code> bin _time span=1h stats count by _time, host</code> : the same buckets as <code>timechart</code> , but as a long table you can process further
<code>strftime(_time, "%H")</code>	Hour of day: <code> eval hour=strftime(_time,"%H") stats count by hour</code> . The busiest hours of the week are 21 (3,039), 20 (2,924) and 19 (2,847).
<code>strftime(_time, "%A")</code>	Day of week: Friday 8,657 (sale day), then Sunday 2,541 and Saturday 2,501
<code>date_hour</code> , <code>date_wday</code>	Default fields taken from the raw timestamp text. They are <i>not</i> adjusted to your time zone and only exist when the timestamp came from the event, so prefer <code>strftime</code> .
<code>stats min(_time) max(_time)</code>	First and last seen; convert with <code>strftime(x, "%d-%b %H:%M:%S")</code> or <code> convert ctime(x)</code>
<code>relative_time(now(), "-1d@d")</code>	Calculate time boundaries inside <code>eval</code> and <code>where</code>

EXAMPLE · ZOOM INTO THE SALE EVENING

```
index=novamart sourcetype=access_combined earliest="09/18/2026:00:00:00" latest="09/19/2026:00:00:00"
| timechart span=1h count
```

Traffic runs at roughly 85–155 requests an hour during the day, then jumps to **1,731** at 19:00 when the sale opens. It peaks at **1,841** at 21:00 and falls to 565 by 23:00. The scraper also shows up as a small bump of 112 at 03:00.

6.5 top, rare and where

top and rare: quick ranking

```
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS
| top limit=2 product_id by region
```

This returns the two best-selling products in each region. In every region they are the **earbuds (NM-1002)** and/or the **Aurora smartphone (NM-1001)**, except South, where the power bank (NM-1003) ties with the earbuds.

Option	Effect
<code>limit=N</code>	How many values (default 10; <code>0</code> = all)
<code>by field</code>	Separate ranking per group
<code>countfield=</code> , <code>percentfield=</code>	Rename the output columns
<code>showperc=false</code> , <code>showcount=false</code>	Hide a column

Option	Effect
<code>useother=true</code>	Add an OTHER row for the remainder

`rare` has the same options but lists the *least* common values. It is useful for spotting anomalies: rare user agents, rare error codes, a host that almost never logs.

where: filter on expressions

`where` keeps only results for which an **eval expression** is true. Use it after `stats` to filter on calculated values, which the `search` command cannot do.

```
index=novamart sourcetype=access_combined
| stats count by clientip
| where count > 300
```

Result: `198.51.100.23` (715, the scraper) and `10.20.0.5` (336, the load-balancer health check). No real shopper comes close.

	search	where
Accepts	Terms, phrases, field=value, wildcards *	Eval expressions: <code>=</code> , <code>≠</code> , <code>></code> , <code>AND</code> , functions
Compare two fields	No	Yes: <code>where errors > total*0.05</code>
String matching	Case-insensitive, * wildcard	Case-sensitive; use <code>like(f, "abc%")</code> or <code>match(f, "regex")</code>
Typical position	Start of the search	After <code>eval</code> or <code>stats</code>

WATCH OUT

In `where`, an unquoted word is treated as a **field name**: `where region=North` compares the field `region` with a field called `North`. Write `where region="North"`.

6.6 Combining eval with Statistical Commands

`eval` can be used **before**, **inside** and **after** a statistics command. Together they turn counts into KPIs.

Before: categorise

```
| eval class=if(status ≥ 500, "error", "ok")
| stats count by class
```

Inside: conditional count

```
| stats count AS total,
count(eval(status ≥ 500))
AS errors
by host
```

The `AS` rename is required.

After: calculate a rate

```
| eval error_pct=
round(errors/total*100,
3)
```

EXAMPLE · CHECKOUT SUCCESS RATE BY PAYMENT METHOD

```
index=novamart sourcetype=novamart:app action=checkout
| stats count AS attempts, count(eval(status="SUCCESS")) AS success by payment_method
| eval success_pct=round(success/attempts*100, 1)
| sort success_pct
```

payment_method	attempts	success	success_pct
NETBANKING	98	84	85.7
CARD	336	292	86.9
WALLET	53	49	92.5
UPI	319	301	94.4
COD	86	86	100.0

CARD and NETBANKING, the two methods that go through gw-beta, are lowest because of the outage.

TIP · EVENTSTATS AND STREAMSTATS

`eventstats` adds an aggregate to every event without collapsing them, e.g. `| eventstats avg(amount) AS avg_amount | where amount > 3*avg_amount` finds unusually large orders. `streamstats` calculates running totals and moving averages in time order. Both are good next steps after this course.

CASE STUDY 5

Did we meet our checkout SLO?

Situation. NovaMart's SRE team has a service-level objective (SLO): *99.5% of checkout requests must not fail with a server error (5xx), measured per day*. The CTO wants to know which days met it and how much of the week's "error budget" the outage burned.

Step 1: Availability per day, with eval inside stats.

```
index=novamart sourcetype=access_combined uri_path=/checkout
| bin _time span=1d
| stats count AS checkouts, count(eval(status ≥ 500)) AS server_errors by _time
| eval availability_pct = round((checkouts - server_errors) / checkouts * 100, 2),
    slo_met = if(availability_pct ≥ 99.5, "yes", "NO")
```

_time	checkouts	server_errors	availability_pct	slo_met
2026-09-14	71	0	100.00	yes
2026-09-15	59	0	100.00	yes
2026-09-16	66	0	100.00	yes
2026-09-17	70	0	100.00	yes
2026-09-18	480	35	92.71	NO
2026-09-19	74	0	100.00	yes
2026-09-20	72	0	100.00	yes

Step 2: The weekly error budget. 892 checkouts × 0.5% allows about **4.5 failures** a week. The outage caused **35**, which burned the budget almost **8 times over**, in 37 minutes, on the busiest evening of the year.

```
index=novamart sourcetype=access_combined uri_path=/checkout
| stats count AS checkouts, count(eval(status≥500)) AS errors
| eval budget=round(checkouts*0.005, 1), budget_used_pct=round(errors/budget*100, 0)
```

Step 3: Where exactly? Five-minute resolution.

```
index=novamart sourcetype=access_combined uri_path=/checkout earliest="09/18/2026:20:00:00" latest="09/18/2026:21:00:00"
| timechart span=5m count AS checkouts, count(eval(status≥500)) AS errors
```

Errors per 5-minute window from 20:05 to 20:40: 3, 3, 5, 7, 2, 7, 3, 5, then zero from 20:45.

Lessons. One `stats` with `eval` inside it gives an SLO report; `timechart` gives the minute-by-minute view. In Module 10 you will turn this exact logic into an alert, and ask whether it would have fired in time.

TRY IT

Lab 8 practises stats, chart, timechart, top, rare and where. **Lab 9** applies them to application performance and error trends.

KNOWLEDGE CHECK · MODULE 6

1. Which command would you use for (a) average payment latency per gateway per hour, drawn as a line chart; (b) revenue per region split by payment method, as a column chart; (c) a table of first-seen and last-seen times per client IP?
2. What is the difference between `count` and `count(error_code)`? And between `count(clientip)` and `dc(clientip)`?
3. Why does `... | stats count by host | where status ≥ 500` return nothing?
4. Rewrite `... | where region=North` so that it works.
5. Write a search that shows, per host, the total requests, the number of 5xx responses and the 5xx percentage.

Lookups and Data Enrichment

Purpose of lookups · CSV lookups and definitions · Automatic vs manual · lookup, inputlookup, outputlookup · Enriching events

AFTER THIS MODULE YOU CAN

- explain what problems lookups solve and which lookup type to use
- upload a CSV lookup, create a lookup definition with advanced matching (CIDR, wildcard, case-insensitive)
- apply lookups manually with `lookup`, and automatically at search time
- read and write lookup tables with `inputlookup` and `outputlookup`
- enrich NovaMart events with product, network, target and threat information

7.1 Why Lookups?

Machine data records *what happened*, but rarely *what it means*. An event says `product_id=NM-1001`, `src_ip=10.20.1.15` or `region=East`. People ask about product **names**, **which network** a login came from, and whether a region is **on target**. A lookup joins external reference data to events **at search time**, without changing the indexed data and without using any licence.

Typical enrichment tables

- Product and price catalogues
- Asset and CMDB lists (host → owner, criticality)
- Network zones (CIDR → zone)
- Error-code and status descriptions
- Threat-intelligence indicators
- Targets, SLAs, thresholds
- Allow/deny lists (known bots, VIP customers)

Lookup types

CSV file	Static or occasionally updated tables. The most common type, and today's focus.
KV Store	Large or frequently updated tables, with per-row updates (stored in the KV Store database).
External	A script is called for each lookup, e.g. DNS resolution.
Geospatial	Map shapes (KMZ) for choropleth maps.

7.2 Creating CSV Lookups and Lookup Definitions

A CSV lookup has three layers. Only the first is mandatory, but the other two make lookups reusable and powerful.

Layer	Where in Splunk Web	What it is
1. Lookup table file	<i>Settings → Lookups → Lookup table files → New</i>	The CSV file itself: a header row of field names, then data rows. Stored in <code>\$SPLUNK_HOME/etc/apps/<app>/lookups/</code> .
2. Lookup definition	<i>Settings → Lookups → Lookup definitions → New</i>	A named reference to the file, plus matching options. Written to <code>transforms.conf</code> .

Layer	Where in Splunk Web	What it is
3. Automatic lookup	<i>Settings → Lookups → Automatic lookups → New</i>	Applies the definition to every search on a source type, source or host. Written to <code>props.conf</code> as <code>LOOKUP-<name></code> .

CSV file rules

- The first row must contain the column names, and they become field names. Use lowercase with underscores (`region_manager`), and match the event field name where possible.
- Keep one row per key value unless you really want multiple matches.
- Encode as UTF-8 and use commas. Quote values that contain commas.
- Newly uploaded files are **private** to you. Share them (*Permissions → This app / All apps*) or colleagues' searches will fail.

Advanced options in the lookup definition

Option (transforms.conf)	Use
<code>match_type = CIDR(cidr)</code>	Match an IP address against network ranges such as <code>10.20.1.0/24</code>
<code>match_type = WILDCARD(uri)</code>	Match patterns such as <code>/api/*</code> in the lookup column
<code>case_sensitive_match = false</code>	Make <code>gw-BETA</code> match <code>gw-beta</code> (matching is case-sensitive by default)
<code>max_matches</code> , <code>min_matches</code>	How many rows may match, and the minimum to return
<code>default_match</code>	Value to output when fewer than <code>min_matches</code> rows match (e.g. <code>unknown</code>)

EXAMPLE · A CIDR LOOKUP FOR NETWORK ZONES

```

NETWORK_ZONES.CSV
cidr,zone,owner,trust
10.20.0.0/24,DMZ - load balancers and web,Platform Team,internal
10.20.1.0/24,Admin workstations (VPN),IT Operations,internal
10.20.2.0/24,Data and payment tier,Payments Team,internal
172.16.0.0/12,Container and lab networks,Lab,internal
192.168.0.0/16,Office LAN,IT Operations,internal

```

TRANSFORMS.CONF (WHAT THE DEFINITION FORM WRITES)

```

[network_zones]
filename = network_zones.csv
match_type = CIDR(cidr)

```

Without `match_type` , Splunk would look for an exact text match of `10.20.1.15` in the `cidr` column and find nothing. With CIDR matching, `10.20.1.15` falls inside `10.20.1.0/24` and gets `zone="Admin workstations (VPN)"` .

7.3 Manual and Automatic Lookups

Manual: the lookup command

```
... | lookup novamart_products product_id
      OUTPUT product_name, category
```

- You choose when and where it runs
- Visible in the search, so easy to understand
- Best while exploring, or when used only in a few searches

Automatic: configured once

```
[novamart:app]
LOOKUP-products = novamart_products
product_id OUTPUT product_name category
```

- Fields appear in every search of the source type, even in the fields sidebar
- Users need no lookup knowledge
- Best for core reference data that everyone needs

TIP

Automatic lookups run at **search time** for every matching search, so keep the tables reasonably small and well keyed. You can filter on automatic lookup output directly, e.g. `index=novamart sourcetype=novamart:app category=Electronics`, because Splunk knows which lookup produces that field.

7.4 lookup, inputlookup and outputlookup

Command	Purpose	Example
<code>lookup</code>	Add fields from a table to each event	<code> lookup gateway_sla gateway OUTPUT sla_p95_latency_ms</code>
<code>inputlookup</code>	Read a table as search results (it is a generating command, so it starts with a pipe)	<code> inputlookup region_targets</code> <code> inputlookup known_bots where policy="block"</code>
<code>outputlookup</code>	Write search results to a table (create or overwrite; <code>append=true</code> adds rows)	<code>... stats count by src_ip outputlookup nm_admin_ips.csv</code>

Syntax details for lookup

```
| lookup <definition> <lookup_field> [AS <event_field>] [OUTPUT | OUTPUTNEW <field> [AS <new_name>] ...]
```

- `AS` after the match field maps a lookup column to a differently named event field: `lookup network_zones cidr AS src_ip OUTPUT zone`.
- `OUTPUT` overwrites existing fields; `OUTPUTNEW` only fills them when they are missing.
- Without an `OUTPUT` clause, **all** other columns are added.
- Events with no match are left without the output fields. Use `fillnull value="Internet" zone` to label them.

Using a lookup as a filter (subsearch)

```
index=novamart sourcetype=access_combined NOT [| inputlookup known_bots | fields clientip]
| stats count
```

The subsearch in square brackets runs first and returns `(clientip="66.249.66.1") OR (clientip="10.20.0.5") OR ...`, which the outer search excludes. Result: **21,371** human requests out of

22,641 (bots and scanners are 5.6% of traffic).

7.5 Enriching Events with External Information

USE CASE · BUSINESS: REVENUE AGAINST TARGET

```
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS
| stats sum(amount) AS revenue by region
| lookup region_targets region OUTPUT region_manager, weekly_revenue_target_inr AS target
| eval attainment_pct = round(revenue / target * 100, 1)
| sort - attainment_pct
```

region	revenue	region_manager	target	attainment_pct
North	1770840	Aditi Rao	1500000	118.1
South	1717583	Karthik Iyer	1600000	107.3
West	996654	Meera Kulkarni	1100000	90.6
East	707421	Sourav Das	800000	88.4

Note the lookup runs *after* `stats`, on 4 rows instead of 812 events. Enriching summarised results is much cheaper.

USE CASE · SRE: LATENCY AGAINST SLA

```
index=novamart sourcetype=novamart:payment
| stats perc95(latency_ms) AS p95_ms by gateway
| lookup gateway_sla gateway OUTPUT provider, sla_p95_latency_ms AS sla_ms
| eval breach = if(p95_ms > sla_ms, "YES", "no")
```

gw-alpha: p95 ≈ 812 ms against an 800 ms SLA, a slight breach worth raising with Payment Partner A.
gw-beta: p95 ≈ 30,257 ms against a 1,000 ms SLA, a massive breach driven entirely by the Friday outage.

USE CASE · WEB ANALYTICS: WHAT ARE PEOPLE LOOKING AT?

```
index=novamart sourcetype=access_combined uri_path=/product.screen
| lookup novamart_products product_id OUTPUT category
| stats count AS views by category | sort - views
```

Electronics 3,062 · Home & Kitchen 1,398 · Fashion 1,265 · Beauty 929 · Sports 656 · Books 521. The 87 views of product IDs that are not in the catalogue (such as the 404 pages for `NM-1099`) get no category and are dropped by `stats ... by category`. Add `| fillnull value="Unknown" category` before `stats` to see them.

CASE STUDY 6

"Is this login from inside?": turning IP addresses into context

Situation. After the bastion break-in, the security lead asks: "Show me every successful SSH login this week by network zone. Any login from outside our networks is an incident." The authentication log contains only IP addresses.

Step 1: Create the CIDR lookup (`network_zones` , `match_type = CIDR(cidr)`), as in 7.2.

Step 2: Enrich and summarise.

```
index=novamart sourcetype=linux_secure "Accepted"
| rex "Accepted (?<auth_method>\S+) for (?<user>\S+) from (?<src_ip>\S+)"
| lookup network_zones cidr AS src_ip OUTPUT zone, trust
| fillnull value="Internet" zone
| fillnull value="external" trust
| stats count, values(user) AS users, values(auth_method) AS methods by zone, trust
```

zone	trust	count	users	methods
Admin workstations (VPN)	internal	76	ananya deploy devops rahul	publickey
Internet	external	1	deploy	password

Step 3: Keep a list of legitimate admin sources (outputlookup).

```
index=novamart sourcetype=linux_secure "Accepted publickey"
| rex "from (?<src_ip>\S+)"
| stats count AS logins, max(_time) AS last_seen by src_ip
| outputlookup nm_admin_ips.csv
```

This writes four rows (10.20.1.15, .22, .31, .44). Next week, any login from an address not in `nm_admin_ips.csv` stands out.

Lessons. One CIDR lookup turned 77 IP addresses into a two-row answer anyone can read, and made the single external password login impossible to miss. Combine it with an alert (Module 10) and the next break-in is caught within minutes.

TRY IT

Lab 10: upload the Day 2 lookups, create definitions including a CIDR match, create an automatic lookup, filter out bots with `inputlookup`, and build a table with `outputlookup`.

KNOWLEDGE CHECK · MODULE 7

1. What are the three layers of a CSV lookup, and which one is required to use the `lookup` command with a friendly name?
2. Your lookup column is called `ip` but your event field is `clientip`. Write the `lookup` command.
3. Why does a network-zones lookup return nothing for `10.20.1.15` until you set `match_type`?
4. When should you prefer an automatic lookup to the `lookup` command?
5. What is the difference between `inputlookup` and `outputlookup`? Give one use of each.
6. A colleague's search using your new lookup fails with "lookup table does not exist". What is the most likely cause?

Reports and Visualisations

Saving searches as reports · Tables and charts · Choosing and configuring visualisations · Scheduling, exporting and sharing · Permissions and ownership

AFTER THIS MODULE YOU CAN

- save a search as a report and manage it from the Reports page
- format statistics tables and choose the right chart for the question
- configure chart options: stacking, axes, legends, overlays, single values
- schedule a report, choose its actions, and export or share the results
- set permissions and ownership correctly, and avoid orphaned reports

8.1 Saving Searches as Reports

A **report** is a saved search plus its presentation: the visualisation, table formatting, time range and, optionally, a schedule. Reports are knowledge objects stored in `savedsearches.conf`.

- 1 Run a search that ends with a **transforming command** (`stats`, `chart`, `timechart`, `top`...) and format it on the **Statistics** or **Visualization** tab.
- 2 **Save As → Report**. Enter a **Title** (use a naming convention, e.g. `NM - Daily Revenue by Category`), a **Description**, the **Content** (Statistics Table or the chart), and whether to show a **Time Range Picker** to viewers.
- 3 From the confirmation dialog, set **Permissions**, **Schedule**, **Acceleration** or **Embed**, **Add to Dashboard**, or **View** the report.

All reports are listed under **Reports** in the app bar. From there you can open, edit (description, permissions, schedule, acceleration), clone, embed or delete them, or *Open in Search* to change the SPL.

TIP · NAMING CONVENTIONS

Prefix knowledge objects with a team or project code and a type: `NM - Report - Daily Revenue`, `NM - Alert - Checkout 5xx`. In a shared Splunk with hundreds of objects, good names are the difference between finding your report and rebuilding it.

8.2 Creating Tables and Charts

Formatting statistics tables

Setting	Where / effect
Rows per page, wrap, row numbers	Format menu above the table
Data overlay	<i>Format → Data Overlay → Heat map</i> (colour by value) or <i>High and low values</i>
Column formatting	Click the paintbrush on a column header: colour by value or ranges, number precision, thousands separator, units (e.g. "₹" or "ms")

Setting	Where / effect
Totals	Add <code> addtotals</code> (row totals) or <code> addcoltotals</code> (column totals) to the SPL
Readable numbers	<code> eval revenue=toString(revenue, "commas")</code> (becomes text, so do this last)

Visualisation types

Compare categories

Column / Bar: revenue by region, errors by host. Bar charts suit long labels.

Trend over time

Line / Area: requests per hour, latency per gateway. Stacked area for parts of a whole over time.

Part of a whole

Pie (few slices only), **stacked column:** payment-method share.

One number

Single value (with trend and sparkline), **radial / filler / marker gauges:** 5xx in the last 15 minutes.

Relationship

Scatter / bubble: order value vs response time.

Location

Cluster map (`iplocation` + `geostats`) and **choropleth** (geospatial lookup) maps.

8.3 Selecting and Configuring Visualisations

Question	SPL shape	Best visualisation
How did daily revenue change, per category?	<code>timechart span=1d sum(amount) by category</code>	Stacked column
Which regions sell the most?	<code>stats sum(amount) by region</code>	Bar (sorted)
Is payment latency healthy right now?	<code>stats avg(latency_ms)</code>	Single value or gauge with colour ranges
Traffic pattern through the day	<code>timechart span=1h count</code>	Line or area
How are payment methods split?	<code>stats count by payment_method</code>	Pie (5 slices) or column
Errors and volume together	<code>timechart count, count(eval(status ≥ 500)) AS errors</code>	Column + chart overlay (errors as a line on a second axis)

Key chart options (Format menu)

Tab	Options
General	Stack mode (not stacked / stacked / stacked 100%), null values (gaps / zero / connect), multi-series mode, drilldown on/off
X-axis	Title, label rotation, truncation
Y-axis	Title, scale (linear / log), interval, min/max. Use a log scale when one series dwarfs the others, as the sale day does here.
Chart overlay	Draw chosen fields as lines, optionally on a second Y-axis
Legend	Position (right, bottom, top, left, none), truncation
Single value	Caption, unit, number precision, trend indicator and sparkline (needs <code>timechart</code>), colour by value with ranges

EXAMPLE · DAILY REVENUE BY CATEGORY, STACKED

```
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS
| lookup novamart_products product_id OUTPUT category
| timechart span=1d sum(amount) by category
```

On the Visualization tab choose **Column**, then *Format* → *General* → *Stack mode: stacked*. On a normal day the stacks are ₹2.8–3.9 lakh. On 18 September the column is ₹30.9 lakh, of which Electronics alone is ₹26.8 lakh. The other days are squashed near zero, so try *Y-axis* → *Scale: Log* to compare them.

8.4 Scheduling, Exporting and Sharing Reports

Scheduling

Reports → *Edit* → *Edit Schedule* (or **Schedule** right after saving):

Setting	Meaning
Schedule	Run every hour, day, week or month at a set time, or Run on Cron Schedule
Time Range	The data window each run covers, e.g. <i>Yesterday</i> or <code>-1d@d</code> to <code>@d</code> . It should match the schedule, so that every event is counted exactly once.
Schedule Priority	Default / Higher / Highest: who runs first when the scheduler is busy
Schedule Window	Lets the scheduler delay a run by up to N minutes to smooth load (not for time-critical reports)
Trigger Actions	Send email (inline table, CSV or PDF), Output results to lookup , Log event , Webhook , run a script, plus actions added by apps

Cron in 30 seconds

```
minute (0-59)
hour (0-23)
day of month (1-31)
month (1-12)
day of week (0-6, 0 = Sunday)

0 8 * * 1      every Monday at 08:00
*/15 * * * *   every 15 minutes
30 6 * * 1-5   06:30 on weekdays
0 */4 * * *    every 4 hours, on the hour
```

Scheduled searches run in the time zone of the Splunk server (or of the report owner, depending on configuration), so check what "08:00" means on your system.

Exporting

- **Export** button (the download icon above the results): CSV, JSON, XML or raw events, with a limit on the number of results.
- **Export** → **Export PDF** and **Print** for reports with visualisations.
- Scheduled email can attach CSV or PDF automatically.
- From the command line, the REST API can export too: `curl -k -u user https://host:8089/services/search/jobs/export -d search="..."`.

Sharing

- Share a report by setting its **permissions** (8.5) and sending its link. Everyone with read access sees it on their Reports page.
- **Embed** (for scheduled reports) gives an iframe snippet for intranet pages. Anyone with the link can see it, so use it with care.
- **Add to Dashboard** turns a report into a dashboard panel that always shows the report's latest results.

8.5 Managing Permissions and Ownership

Sharing level	Who can see it
Private (Owner)	Only the owner (and admins). The default for everything you create.
This app only	Users of the app the object lives in, subject to role-based read/write
All apps (Global)	Users in every app, subject to role-based read/write

- Once shared, you grant **Read** and **Write** per role, e.g. read for *user*, write for *power* and *admin*.
- **Run As: Owner / User.** A shared scheduled report runs with the *owner's* permissions by default, so viewers see results from indexes the owner can see. Choose *User* when viewers must only see data their own role allows.
- **Ownership.** Every object belongs to a user. When that user leaves and their account is deleted, their scheduled searches become **orphaned** and stop running. Reassign them first: *Settings* → *All configurations* → *Reassign Knowledge Objects*.

WATCH OUT · RUN AS OWNER CAN LEAK DATA

If an admin shares a report of `index=novamart sourcetype=linux_secure` with *Run As: Owner*, every user who can read the report sees the security events, even users whose role cannot search that data. Review Run As whenever you share reports that contain sensitive data.

CASE STUDY 7

The Monday management pack

Situation. NovaMart's leadership wants a short revenue and operations summary *in their inbox every Monday at 08:00*, covering the previous Monday to Sunday. The finance director also wants the raw numbers in a CSV so she can reuse them in her spreadsheets.

Design.

Decision	Choice and reason
Search	<code>... status=SUCCESS lookup novamart_products product_id OUTPUT category stats count AS orders, sum(amount) AS revenue by category addcoltotals labelfield=category label="TOTAL"</code>
Schedule	Cron <code>0 8 * * 1</code> (Mondays 08:00)
Time range	Earliest <code>-7d@w1</code> , latest <code>@w1</code> : exactly the previous Monday 00:00 to this Monday 00:00, so no overlap and no gap
Actions	Send email to leadership: inline table, PDF attached, CSV attached for finance. Also <i>Output results to lookup</i> (<code>nm_weekly_revenue.csv</code>) to keep a history for trend dashboards.

Decision	Choice and reason
Permissions	App-level, read for the <i>nm_business</i> role; owner = a service account, not a person, so it is never orphaned
Run As	Owner is fine: the report contains no sensitive data

What leadership would have received for 14–20 September: 812 orders, ₹51,92,498 revenue, of which Electronics ₹42,27,221 (81%).

Lessons. A good scheduled report is 20% SPL and 80% decisions: the time range that matches the schedule, who receives it and in what format, who owns it, and whose permissions it runs with.

TRY IT

Lab 11: build two reports, format a table and a chart, schedule one of them with *Output results to lookup* (so you can verify that it ran), export results, and set permissions.

KNOWLEDGE CHECK · MODULE 8

1. Which visualisation would you use for (a) hourly requests over a day, (b) the current number of 5xx errors, (c) revenue by category per day?
2. Write a cron expression for "every weekday at 07:30".
3. A daily report runs at 06:00 over "Last 24 hours". Why might totals differ from a report that uses `-1d@d` to `@d`? Which is better for daily business totals?
4. What happens to a user's scheduled reports when their account is deleted? How do you prevent it?
5. Explain the security risk of sharing a report with *Run As: Owner*.

Dashboards

Creating dashboards and search-based panels · Tables, charts and single values · Inputs, tokens and time-range controls · Interactivity, arrangement and sharing

AFTER THIS MODULE YOU CAN

- explain the difference between Dashboard Studio and Classic (Simple XML) dashboards
- build panels from inline searches and saved reports
- add a time-range picker and dropdown inputs, and use their tokens in searches
- add drilldown interactivity, arrange panels, and share the dashboard
- apply design and performance good practice

9.1 Dashboard Frameworks

	Dashboard Studio	Classic (Simple XML)
Definition format	JSON (<i>Source</i> view)	XML (<i>Edit</i> → <i>Source</i>)
Layout	Grid (auto-arranged rows) or Absolute (pixel placement, images, shapes, backgrounds)	Rows and panels
Look	Modern, highly customisable; good for NOC screens and executive views	Standard look, simpler
Data sources	Inline search, saved search (report), chain searches (base + post-process)	Inline search, report, base search + post-process
Status	Splunk's recommended framework for new dashboards	Still widely used; many existing dashboards and apps

The concepts are the same in both: **panels** show the results of **searches**, **inputs** set **tokens**, and tokens are substituted into searches. This module uses Dashboard Studio for building, and shows the Classic XML equivalent where it helps. Lab 12's appendix contains a full Classic version of the lab dashboard.

9.2 Creating Dashboards and Search-Based Panels

- 1 **Create:** *Dashboards* → *Create New Dashboard*. Enter a title (the ID is generated from it), a description and permissions, and choose **Dashboard Studio** with **Grid** layout.
- 2 **Add a panel:** click + (**Add chart**), choose a visualisation type, then **Create search** (inline SPL), or *Select a saved search* to reuse a report.
- 3 **Configure:** in the right-hand configuration panel, set the title, the data source, and display options (colours, units, legend, axes).
- 4 **Save**, then **View**. Everything you clicked is stored in the JSON definition (*Source* button), which you can copy, version-control and paste into another Splunk.

TIP · ANOTHER QUICK WAY

From any search: **Save As** → **Dashboard Panel** → **New/Existing Dashboard**. This is the fastest way to turn a working search into a panel.

Panel data sources

Source	When to use	Note
Inline search	Panel-specific searches	Runs every time the dashboard loads
Saved report	Heavy searches that many people view	A <i>scheduled</i> report shows its last results instantly, with no new search
Base + chain (post-process)	Several panels built from the same events	One search reads the index; each panel adds its own <code> stats ...</code> . This is far cheaper.

9.3 Tables, Charts and Single-Value Panels

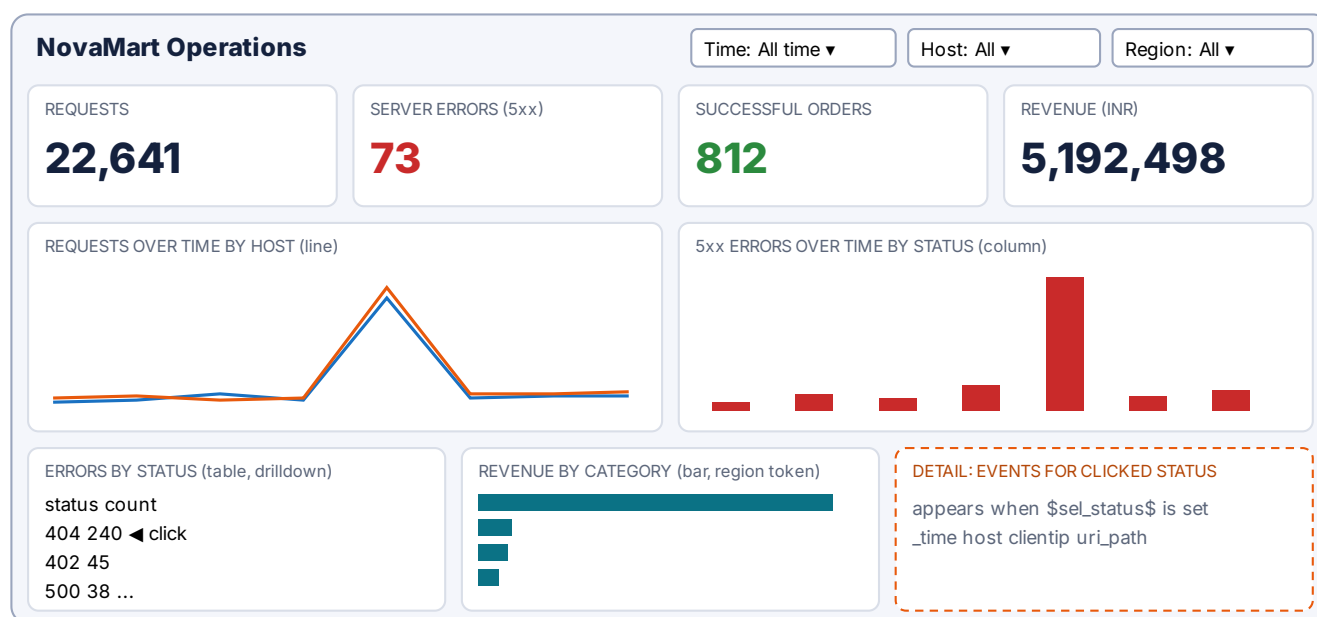


Figure 9.1 — Wireframe of the NovaMart Operations dashboard you will build in Lab 12 (values shown for All time, all hosts)

Panel type	Search pattern	Configuration tips
Single value	<code>... stats count</code> or <code>... timechart count</code> (for a trend and sparkline)	Add a unit, and colour ranges such as green 0–4, amber 5–19, red 20+
Line / area	<code>timechart span=1h count by host</code>	Keep the number of series small; set the null-value mode
Column / bar	<code>stats sum(amount) by category sort - sum(amount)</code>	Sort the data; use bars for long labels
Table	<code>stats count by status sort - count</code>	Format columns; enable drilldown
Pie	<code>stats count by payment_method</code>	Five slices at most

9.4 Inputs, Tokens and Time-Range Controls

An **input** is a control on the dashboard (time picker, dropdown, multiselect, text box, radio buttons). Each input sets a **token**: a named variable. Any search that contains `$token$` re-runs with the new value whenever the user changes the input.

Input	Token(s)	Used in SPL like
Time range	<code>\$global_time.earliest\$</code> , <code>\$global_time.latest\$</code>	Applied as the panel's time range. In Studio, the default ("global") time input applies to all searches.
Dropdown (static)	<code>\$host\$</code> with choices All= *, web01, web02, web03	<code>index=novamart sourcetype=access_combined host=\$host\$</code>
Dropdown (dynamic)	<code>\$region\$</code> , values from a search: <code>... stats count by region</code>	<code>... region=\$region\$</code>
Text	<code>\$order\$</code>	<code>index=novamart "\$order\$"</code>
Multiselect	<code>\$methods\$</code> with prefix/suffix/delimiter settings	<code>payment_method IN (\$methods\$)</code>

EXAMPLE · THE SAME PANEL, DIFFERENT TOKEN VALUES

```
index=novamart sourcetype=access_combined host=$host$ status>=500 | stats count
```

\$host\$	requests (other panel)	5xx count
* (All)	22641	73
web01	10856	43
web02	11785	30

Historical data, All time. With the live generator running, the host `web03` adds live events.

WATCH OUT · DEFAULT VALUES

Always give inputs a **default** (e.g. * for All). Without one, panels that use the token show "Waiting for input..." until the user picks a value. Also consider how tokens are quoted: `host="$host$"` is safer when values may contain spaces.

Classic (Simple XML) equivalent

SIMPLE XML: A TIME PICKER AND A HOST DROPDOWN DRIVING A PANEL

```
<form version="1.1">
  <label>NovaMart Operations</label>
  <fieldset submitButton="false">
    <input type="time" token="t"><label>Time</label><default><earliest>0</earliest><latest></latest></default></input>
    <input type="dropdown" token="host"><label>Host</label>
      <choice value="*">All</choice><choice value="web01">web01</choice><choice value="web02">web02</choice>
    <default>*</default></input>
  </fieldset>
  <row><panel><title>Server errors</title>
    <single><search><query>index=novamart sourcetype=access_combined host=$host$ status>=500 | stats count</query>
      <earliest>${t.earliest}</earliest><latest>${t.latest}</latest></search></single>
    </panel></row>
</form>
```

In XML, `>` and `<` inside queries must be escaped as `>` and `<`, or the query wrapped in `<![CDATA[ ... ]]>`.

9.5 Interactivity, Arrangement and Sharing

Drilldown

Clicking a chart or table can do one of four things:

Drilldown action	Example
Open the search (default)	Click a bar and see the underlying events in the Search app
Set a token	Click status <code>404</code> in the errors table → <code>\$sel_status\$=404</code> → a detail panel shows those events
Link to a dashboard	Pass the clicked host to a "Host detail" dashboard: <code>?form.host=\$row.host.value\$</code>
Custom URL	Open a ticket in the ITSM tool, or a runbook page, with the clicked value

In Studio: select the panel → **Interactions** → **+ Add Interaction** → **Set tokens**. Set *Token name* `sel_status` and *Token value* **Name of clicked item**, **Value of clicked item** or a specific column, e.g. `row.status.value`.

In Classic XML: `<drilldown><set token="sel_status">$row.status$</set></drilldown>`, and `depends="$sel_status$"` on the detail panel so that it only appears after a click.

Arrangement and design good practice

Do

- Design for one audience and one purpose (NOC ≠ executive)
- Put the KPIs (single values) at the top left; details below
- Use one global time range for all panels
- Use colour only for meaning (red = bad), and consistently
- Use chain (base) searches or scheduled reports for heavy panels
- Keep it to about 6–12 panels per dashboard; link to detail dashboards

Avoid

- Real-time searches on every panel (costly)
- All-time searches on large indexes by default
- Pie charts with many slices; 3D anything
- Panels that nobody acts on
- Hard-coded values where a token belongs

Sharing

- **Permissions** work exactly as for reports: private, app or global; read/write per role.
- **Export:** Studio has *Download PDF / PNG*; Classic has *Export PDF* and **Schedule PDF Delivery** by email.
- **Home dashboard:** users can set a dashboard as their home page.
- **Refresh:** set an automatic refresh interval (e.g. every 60 s) for wall screens that show live data.

CASE STUDY 8

A NOC screen that would have caught Friday's outage

Situation. NovaMart's Network Operations Centre has a wall screen. The on-call lead wants a dashboard that answers, in ten seconds: *Is the site up? Are customers failing to pay? Is anyone attacking us?*

Design decisions.

Question	Panel	Why
Is the site up?	Single value: requests in the last 15 min, with a trend sparkline	A sudden drop means an outage upstream
Are payments failing?	Single value: checkout 5xx in the last 15 min, red at 5 or more	Colour does the talking on a wall screen
Which gateway?	Line: avg <code>latency_ms</code> by gateway, 5-minute span	gw-beta jumped from under 700 ms to ~30,000 ms at 20:05 on sale night
Under attack?	Table: top 5 source IPs with failed SSH passwords	203.0.113.77 would top the list at 02:11
Business impact	Single value: successful orders in the last 15 min	Shows the effect on the business, not only the technical symptom

Replaying Friday. Set the time range to 18 Sep, 20:00–21:00, with 5-minute spans. From 20:05 the gw-beta latency line leaves the chart (~30,000 ms in every 5-minute bucket to 20:40). The checkout-5xx single value turns red from the 20:15 window (5 errors). The screen would have shown the problem within minutes, *if someone was looking*. That caveat is why Module 10 adds alerts.

TRY IT

Lab 12: build the NovaMart Operations dashboard in Dashboard Studio: four KPI single values, two charts, a drilldown table, time/host/region inputs, then watch it update with live data.

KNOWLEDGE CHECK · MODULE 9

1. Name two differences between Dashboard Studio and Classic dashboards.
2. What is a token? Give an example of a token set by an input and one set by a drilldown.
3. Why should a dropdown input have a default value such as `*`?
4. Five panels all start with `index=novamart sourcetype=access_combined`. How can you make the dashboard load faster?
5. In Simple XML, why does `status ≥ 500` inside a `<query>` break the dashboard, and how do you fix it?

Alerts and Monitoring

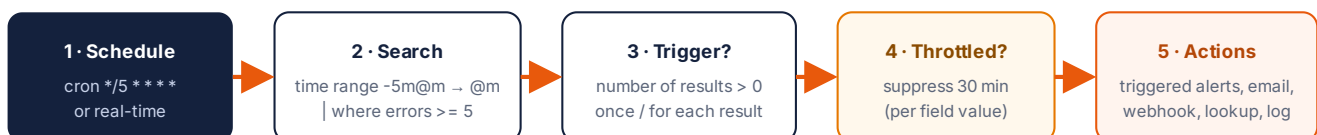
Scheduled and real-time alerts · Trigger conditions, throttling and actions · Email and webhook · Alert management, troubleshooting and best practice

AFTER THIS MODULE YOU CAN

- explain how an alert works and choose between scheduled and real-time alerts
- set trigger conditions, trigger modes and throttling
- configure email and webhook actions, using tokens in messages
- manage alerts, find out why an alert did or did not fire, and apply best practice

10.1 How an Alert Works

An alert is a **saved search** that runs on a **schedule** (or continuously), checks its results against a **trigger condition**, and, if the condition is met and the alert is not **throttled**, runs one or more **actions**.



Stops at step 3 if the condition is not met, and at step 4 if a recent trigger is still being suppressed

Figure 10.1 — The alert pipeline

Scheduled vs real-time alerts

	Scheduled	Real-time
How it runs	Every N minutes or hours (cron), over a fixed window	Continuously: per result (each matching event) or over a rolling window (e.g. the last 5 minutes)
Latency	Up to one schedule interval, plus the run time	Seconds
Cost	Low: runs briefly, then stops	High : a search runs all the time on the search head and indexers
Use for	Almost everything: error thresholds, trends, daily checks	Rare, critical, per-event cases, e.g. a root login from the Internet

TIP

A scheduled alert every 1–5 minutes is "near real-time" for most operational needs, at a fraction of the cost. Many organisations restrict the `rtsearch` capability to administrators.

10.2 Trigger Conditions, Throttling and Alert Actions

Creating an alert

Run the search, then **Save As** → **Alert**:

Field	What to set
Title, Description	e.g. <code>NM - Alert - Checkout 5xx spike</code> ; the description says what it means and what to do
Permissions	Private or shared in the app
Alert type	Scheduled (run every hour/day/week/month, or <i>Run on Cron Schedule</i> with a time range) or Real-time
Expires	How long the triggered-alert record (and its results) is kept, e.g. 24 hours
Trigger conditions	<i>Number of Results / Hosts / Sources</i> is greater than, less than, equal to... N; <i>Custom</i> (a search condition such as <code>search errors > 10</code>); or <i>Per-Result</i> (real-time)
Trigger	Once (one action for the whole result set) or For each result (one action per row, e.g. per attacking IP)
Throttle	Suppress triggering for a period; with "For each result", suppress per field value (e.g. <code>src_ip</code>)
Trigger actions	Add to Triggered Alerts (with severity), Send email, Webhook, Log Event, Output results to lookup, Run a script, plus app-provided actions (Slack, PagerDuty, ServiceNow, SOAR...)

TIP · PUT THE THRESHOLD IN THE SPL

Instead of "number of results > 37", end the search with `| stats count AS errors | where errors ≥ 5` and trigger on *number of results* > 0. The logic is then visible in the search, testable by running it, and the result row carries the numbers into the alert message.

Throttling

Without throttling, a scheduled alert that runs every 5 minutes during a 40-minute outage fires up to 8 times. Throttling suppresses repeats:

- **Trigger once + throttle 30 minutes:** one notification per 30 minutes, whatever happens.
- **For each result + throttle on `src_ip` for 60 minutes:** one notification per attacking IP per hour. A second attacker is still reported straight away.

Choosing the time window

The alert's time range should equal its schedule interval, snapped to whole minutes, so that every event is checked exactly once:

Schedule	Time range	Why
<code>*/5 * * * *</code>	<code>-5m@m</code> to <code>@m</code>	No gaps, no double-counting
<code>*/5 * * * *</code>	<code>-6m@m</code> to <code>-1m@m</code>	Same, but allows one minute for forwarding and indexing delay, so late events are not missed
<code>0 * * * *</code>	<code>-1h@h</code> to <code>@h</code>	Hourly checks

10.3 Email and Webhook Overview

Email

- An administrator first configures the mail server: *Settings* → *Server settings* → *Email settings* (SMTP host and port, TLS/SSL, user name/password, "Send emails as", link hostname).
- Per alert: To/Cc/Bcc, priority, subject, message, and whether to include the results inline or as CSV/PDF attachments, the link to the alert and the search.
- Messages can use **tokens**:

Token	Value
<code>\$name\$</code>	The alert title
<code>\$trigger_time\$</code> , <code>\$trigger_date\$</code>	When it fired
<code>\$result.errors\$</code>	Value of the field <code>errors</code> in the first result row
<code>\$job.resultCount\$</code>	Number of result rows
<code>\$results_link\$</code>	Link to the results in Splunk
<code>\$app\$</code> , <code>\$owner\$</code> , <code>\$search\$</code>	Context

EXAMPLE SUBJECT AND MESSAGE

Subject: [SEV-2] `$name$` - `$result.errors$` checkout errors in 5 min

Message: Checkout is failing. `$result.errors$` HTTP 5xx on /checkout between `$result.window_start$` and `$result.window_end$`

affecting `$result.customers$` customers.

Runbook: <https://wiki.novamart.example/runbooks/checkout-5xx>

Results: `$results_link$`

Webhook

The webhook action POSTs a JSON document to a URL, which is how Splunk talks to chat tools, incident platforms and automation. The payload looks like this:

WEBHOOK PAYLOAD (ABRIDGED)

```
{
  "sid": "scheduler__admin__search__RMD5...",
  "search_name": "NM - Alert - Checkout 5xx spike",
  "app": "search",
  "owner": "admin",
  "results_link": "http://splunk:8000/app/search/@go?sid=scheduler__admin__search__RMD5...",
  "result": { "errors": "37", "customers": "37", "window_start": "20:15:00", ... }
}
```

Only the **first** result row is included in `result`. Some Splunk versions and Splunk Cloud Platform restrict webhook destinations to an allow list (*Settings* → *Alert actions* → *Webhook*); add your URL there if calls are blocked. In Lab 13 you will receive these payloads with `tools/webhook_listener.py`.

10.4 Alert Management, Troubleshooting and Best Practice

Where to manage alerts

Place	Use
Alerts (app bar)	List, edit, enable/disable, clone and delete your alerts

Place	Use
Activity → Triggered Alerts	History of alerts that fired and added themselves to Triggered Alerts, with severity and links
Settings → Searches, reports, and alerts	All saved searches across apps and owners; change owner, schedule and permissions

"Why didn't my alert fire?": troubleshooting with `_internal`

1. DID IT RUN, AND WHAT DID IT FIND?

```
index=_internal sourcetype=scheduler savedsearch_name="NM - Alert - Checkout 5xx spike"
| table _time, status, run_time, result_count, alert_actions, suppressed
```

2. DID THE ACTION SUCCEED? (EMAIL, WEBHOOK...)

```
index=_internal sourcetype=splunkd component=sendmodalert
| table _time, action, log_level, event_message
```

```
index=_internal source=*python.log* sendemail ERROR
```

3. IS THE SCHEDULER OVERLOADED? (SKIPPED SEARCHES)

```
index=_internal sourcetype=scheduler status=skipped | stats count by savedsearch_name, reason
```

Symptom	Likely cause
No scheduler entries	The alert is disabled, not scheduled, or its owner no longer exists (orphaned)
<code>result_count=0</code>	The condition was not met: check the time range, the threshold, and indexing delay
Ran and triggered but no email	The mail server is not configured; see the <code>sendemail</code> errors in <code>python.log</code>
<code>suppressed=1</code>	Throttling is working as designed
<code>status=skipped</code>	Too many concurrent scheduled searches: spread schedules, or raise the priority

Best practice

Design

- Alert on **symptoms users feel** (checkout errors) before internal causes
- Prefer **rates** (errors ÷ total) over raw counts when traffic varies
- Every alert needs an owner, a severity, and a runbook link in the message
- Backtest**: run the alert search over past incidents and quiet periods before enabling it

Operate

- Throttle, and tune thresholds to avoid **alert fatigue**
- Avoid real-time alerts unless per-event speed is essential
- Stagger cron minutes (e.g. `2-59/5`) to avoid everything running at :00
- Review monthly: which alerts fired, which were acted on, which were noise

CASE STUDY 9

Would we have caught it? Backtesting two alerts on last week's data

Situation. Before enabling new alerts, the SRE and security leads ask: "If these had existed last Friday and Thursday, when would we have known?" Backtesting answers that by running the alert logic over the historical incident windows.

Alert A: checkout server errors. Rule: 5 or more checkout 5xx in a 5-minute window.

```
index=novamart sourcetype=access_combined uri_path=/checkout status>=500
  earliest="09/18/2026:19:00:00" latest="09/19/2026:00:00:00"
| bin _time span=5m
| stats count AS errors by _time
| eval would_trigger=if(errors>=5, "YES", "no")
```

window	20:05	20:10	20:15	20:20	20:25	20:30	20:35	20:40
errors	3	3	5	7	2	7	3	5
≥5?	no	no	YES	YES	no	YES	no	YES

With threshold 5, the first alert fires after the 20:15–20:20 window, at about **20:20: 12½ minutes** after the first customer error (20:07:38). With threshold 3 it would fire at **20:10**, only 2½ minutes after, but with a higher risk of noise on normal days. (No normal day had a single checkout 5xx, so 3 would be safe with this data.) Throttling for 30 minutes turns four triggers into **one** notification.

Even earlier: the cause. An alert on `"Circuit breaker" gw-beta` WARN events would have fired at **20:05:01**, before any customer saw an error. Symptom alerts (checkout errors) should come first, and cause alerts (circuit breakers) can add early warning.

Alert B: SSH brute force. Rule: more than 20 failed passwords from one source IP in 5 minutes; trigger for each result, throttled per `src_ip` for 60 minutes.

```
index=novamart sourcetype=linux_secure "Failed password"
| rex "Failed password for (invalid user )?(?<user>\S+) from (?<src_ip>\d{1,3}(\.?\d{1,3}){3})"
| bin _time span=5m
| stats count AS failures, dc(user) AS users by _time, src_ip
| where failures > 20
```

Result: five windows, all for `203.0.113.77` on 17 September (02:10: 73, 02:15: 83, 02:20: 87, 02:25: 88, 02:30: 83). No other IP ever exceeded 1 failure in 5 minutes, so the threshold creates no noise. The alert would have fired at about **02:15, 20 minutes before the attacker got in** (02:34:51), and throttling would have sent one notification rather than five.

Lessons. Backtesting turns "we think this alert is good" into evidence: how fast it fires, and how often it would have been noise. Both alerts are built in Lab 13 and tested with live generated traffic.

TRY IT

Lab 13: start the live traffic generator, create a threshold-based checkout alert with throttling and a webhook action, trigger it with an injected incident, then create a per-result brute-force alert and troubleshoot both with `_internal`.

KNOWLEDGE CHECK · MODULE 10

1. When would you choose a real-time alert over a scheduled one? What is the cost?
2. An alert runs every 5 minutes over "Last 15 minutes". What problem does this cause, and how do you fix it?
3. What is the difference between trigger *Once* and *For each result*? Which suits a brute-force alert, and why?
4. Which token puts the value of the field `errors` from the first result into an email subject?
5. An alert "never fires". List the three `_internal` checks you would make, in order.

Basic Splunk Administration

Configuration directories · inputs, outputs, props, transforms · Indexes and retention · Roles, capabilities and search permissions · Monitoring Console · Troubleshooting with `_internal`

AFTER THIS MODULE YOU CAN

- find your way around `$SPLUNK_HOME` and explain configuration precedence
- read and write basic stanzas in `inputs.conf`, `outputs.conf`, `props.conf` and `transforms.conf`
- create an index and set its retention, and estimate storage
- create a role with limited index access and a search filter, and assign it to a user
- use the Monitoring Console and `_internal` searches to troubleshoot

11.1 Configuration Directory Structure

Everything Splunk does is controlled by plain-text `.conf` files. Splunk Web, the CLI and the REST API are just different ways of editing them. Knowing where they live is the foundation of administration.

```
$SPLUNK_HOME (E.G. /OPT/SPLUNK, OR /HOME/UBUNTU/SPLUNK ON YOUR LAB SERVER)
├─ bin/                               splunk CLI, splunkd, python
├─ etc/
│   ├── system/
│   │   ├── default/                 Splunk's shipped defaults      ← NEVER edit
│   │   └─ local/                   system-wide overrides
│   └─ apps/
│       ├── search/                 the Search & Reporting app
│       │   ├── default/             shipped with the app          ← never edit
│       │   ├── local/               your changes made in this app (UI writes here)
│       │   ├── lookups/             CSV lookup files
│       │   └─ metadata/             default.meta / local.meta (permissions)
│       └─ novamart_lab/             our lab app (default/, lookups/, data/ ...)
│   ├── users/<user>/<app>/local/    private knowledge objects per user
│   └─ deployment-apps/             apps the deployment server pushes to clients
└─ var/
    ├── lib/splunk/                 index data (defaultdb = main, novamart/, ...), fishbucket
    └─ log/splunk/                 splunkd.log, metrics.log, scheduler.log, license_usage.log ...
```

default vs local, and precedence

- `default/` holds what Splunk or an app developer ships. It is overwritten on upgrade, so never edit it.
- `local/` holds your changes. Only the settings you change need to be in `local`; Splunk merges the layers.
- When the same setting appears in several places, **precedence** decides which one wins:

Context	Applies to	Highest → lowest
Global (index-time, inputs, server)	inputs, outputs, indexes, index-time props/transforms	<code>system/local</code> → <code>apps/*/local</code> → <code>apps/*/default</code> → <code>system/default</code>
App/user (search-time)	field extractions, lookups, saved searches, event types	<code>users/<you>/<app>/local</code> → current app <code>local</code> → current app <code>default</code> → other apps (if exported) → <code>system</code>

btool: see the effective configuration

```
$SPLUNK_HOME/bin/splunk btool inputs list --debug | grep novamart      # which file each setting came from
$SPLUNK_HOME/bin/splunk btool props list novamart:app --debug
$SPLUNK_HOME/bin/splunk btool indexes list novamart
$SPLUNK_HOME/bin/splunk btool check                                  # syntax problems in .conf files
```

WATCH OUT · RESTART OR NOT?

Changes made in Splunk Web mostly apply immediately. Changes to `.conf` files made by hand usually need a **restart** (`splunk restart`), especially inputs, indexes, outputs and index-time props/transforms. Search-time changes can often be reloaded with

`http://<server>:8000/debug/refresh`.

11.2 inputs.conf, outputs.conf, props.conf and transforms.conf

File	Controls	Lives on	Pipeline phase
<code>inputs.conf</code>	What data to collect: files, ports, scripts, HEC; with index, source type and host	Forwarders, indexers, heavy forwarders	Input
<code>outputs.conf</code>	Where forwarders send data (indexer groups, load balancing, TLS, acknowledgement)	Forwarders	Output
<code>props.conf</code>	Per source type / source / host: line breaking, timestamps, <code>SEDCMD</code> , and pointers to transforms (index time); <code>EXTRACT-</code> , <code>EVAL-</code> , <code>LOOKUP-</code> , <code>KV_MODE</code> (search time)	Indexers/HF (index time); search heads (search time)	Parsing + search
<code>transforms.conf</code>	The detailed rules props refers to: filter to <code>nullQueue</code> , route to index, override host/source type, mask; lookup definitions; <code>REPORT-</code> extractions	Same as props	Parsing + search

Stanza syntax

```
# Comments start with # and must be on their own line (not after a value)
# Stanza examples: [monitor:///var/log/secure] [novamart:app] [source::...secure.log] [host::web*]
[stanza_name]
setting = value
```

EXAMPLE · DROP NOVAMART'S HEALTH CHECKS BEFORE THEY ARE INDEXED

The load balancer's `GET /health` requests (336 events a week here, millions in production) cost licence and add nothing to analysis. Two stanzas send them to the `nullQueue` (the bin) at parse time:

```
PROPS.CONF (INDEXER OR HF)
[access_combined]
TRANSFORMS-drop_health = nm_drop_health
SEDCMD-mask_session = s/JSESSIONID=\w+/JSESSIO
NID=XXXX/g
```

```
TRANSFORMS.CONF
[nm_drop_health]
REGEX = "GET /health HTTP
DEST_KEY = queue
FORMAT = nullQueue
```

The `SEDCMD` line also masks session IDs before storage (a privacy control). Both apply only to data indexed *after* the change and restart.

EXAMPLE · ROUTE SECURITY DATA TO ITS OWN INDEX

```
PROPS.CONF
[linux_secure]
TRANSFORMS-route_sec = nm_route_security
```

```
TRANSFORMS.CONF
[nm_route_security]
REGEX = .
DEST_KEY = _MetaData:Index
FORMAT = security
```

Every `linux_secure` event goes to index `security`, which can have longer retention and stricter access than `novamart`.

11.3 Index and Retention Concepts

Each index is a set of directories of **buckets** (Day 1: hot → warm → cold → frozen). `indexes.conf` controls where they live and how long data is kept.

Setting	Default	Meaning
<code>homePath</code> / <code>coldPath</code> / <code>thawedPath</code>	<code>\$SPLUNK_DB/<index>/db ...</code>	Where hot/warm, cold and thawed buckets are stored
<code>frozenTimePeriodInSecs</code>	188697600 (≈ 6 years)	Time-based retention: a bucket freezes when its <i>newest</i> event is older than this
<code>maxTotalDataSizeMB</code>	500000 (≈ 488 GB)	Size-based retention: the oldest buckets freeze when the index exceeds this
<code>maxDataSize</code>	<code>auto</code> (750 MB)	Size of a hot bucket before it rolls to warm (<code>auto_high_volume</code> = 10 GB for busy indexes)
<code>coldToFrozenDir</code>	not set	If set, frozen buckets are archived here instead of deleted

WATCH OUT · WHICHEVER COMES FIRST

Data is frozen when *either* limit is reached. Frozen data is **deleted by default**. If regulations require 1 year of authentication logs, check both the time *and* the size limit, and consider `coldToFrozenDir`.

EXAMPLE · AN INDEX FOR THE NEW WAREHOUSE FEED, KEPT FOR ONE YEAR

```
[novamart_inv]
homePath  = $SPLUNK_DB/novamart_inv/db
coldPath  = $SPLUNK_DB/novamart_inv/colddb
thawedPath = $SPLUNK_DB/novamart_inv/thaweddb
# 365 days x 86,400 s
frozenTimePeriodInSecs = 31536000
# 5 GB cap
maxTotalDataSizeMB = 5000
```

Sizing rule of thumb: disk $\approx$ daily raw volume $\times$ retention days $\times$ ~50% (compressed raw data $\approx$ 15% plus index files $\approx$ 35%). For NovaMart production at 110 GB/day kept 90 days: $110 \times 90 \times 0.5 \approx 4.95$ TB, before replication in a cluster.

INSPECT INDEXES

```
| eventcount summarize=false index=novamart index=novamart_inv
| dbinspect index=novamart | table bucketId, state, startEpoch, endEpoch, eventCount, sizeOnDiskMB
| rest /services/data/indexes | table title, frozenTimePeriodInSecs, maxTotalDataSizeMB, currentDBSizeMB,
totalEventCount
```

11.4 User Roles, Capabilities and Search Permissions

Splunk uses **role-based access control (RBAC)**. Users get one or more roles; roles grant **capabilities** (what you can *do*) and **index access** (what data you can see).

Built-in role	Typical use
<code>user</code>	Search, create own reports and dashboards, schedule searches
<code>power</code>	user + share knowledge objects with others, real-time search, edit shared objects
<code>admin</code>	Everything: users, inputs, indexes, apps, settings
<code>can_delete</code>	Only the <code>delete</code> command (hide events). Grant temporarily and deliberately.

Role setting (Settings → Roles)	Meaning
Inheritance	Start from another role (e.g. <code>user</code>) and add to it
Capabilities	e.g. <code>schedule_search</code> , <code>rtsearch</code> , <code>edit_user</code> , <code>admin_all_objects</code> , <code>delete_by_keyword</code>
Indexes: <i>Included</i> (<code>srchIndexesAllowed</code>)	Indexes the role may search
Indexes: <i>Default</i> (<code>srchIndexesDefault</code>)	Searched when no <code>index=</code> is given
Restrictions: search filter (<code>srchFilter</code>)	An extra filter silently added to every search, e.g. <code>NOT sourcetype=linux_secure</code>
Resources	Concurrent search jobs quota, disk quota, time-window limit, real-time quota

EXAMPLE · A BUSINESS ANALYST ROLE THAT CANNOT SEE SECURITY DATA

```
AUTHORIZE.CONF (WHAT SETTINGS → ROLES WRITES)
[role_nm_analyst]
importRoles = user
srchIndexesAllowed = novamart
srchIndexesDefault = novamart
srchFilter = NOT sourcetype=linux_secure
srchJobsQuota = 5
```

A user with only `nm_analyst` who runs `index=novamart | stats count by sourcetype` sees `access_combined`, `novamart:app` and `novamart:payment`, but **no** `linux_secure`, and cannot search `novamart_inv` or `_internal` at all. Remember the Module 8 warning: a report shared with *Run As: Owner* by an admin bypasses this filter.

Authentication can be native (Splunk users), LDAP/Active Directory, or SAML single sign-on. With LDAP or SAML, you map directory groups to Splunk roles instead of assigning roles user by user.

11.5 Monitoring Console Overview

The **Monitoring Console (MC)** is a built-in app of dashboards about Splunk itself: *Settings* → *Monitoring Console*. On a single server it runs in **standalone** mode; in a distributed deployment, it is configured once (*Settings* → *General Setup*) to know every instance and its role.

MC area	Answers
Overview	Health of each instance at a glance
Indexing → Performance, Inputs, Indexes and Volumes	Throughput by source type/index, blocked queues , index sizes and retention
Search → Activity, Scheduler	Who is running what, slow searches, skipped scheduled searches
Resource Usage	CPU, memory and disk by process and search
Forwarders	Which forwarders are connected or missing (enable forwarder monitoring first)
Licensing	Daily usage by index, source type, host and pool
Health Check	A set of best-practice checks you can run on demand

Also check the **health report** icon in the Splunk bar (green/yellow/red), which summarises the status of splunkd features such as data forwarding, indexing queues and the search scheduler.

11.6 Troubleshooting Using `_internal` Logs

Source in <code>_internal</code>	Contains
<code>splunkd.log</code>	Main daemon log: <code>log_level</code> , <code>component</code> (TailReader, DateParserVerbose, AggregatorMiningProcessor, TcpOutputProc...)
<code>metrics.log</code>	Every 30 s: throughput per index/source type/host, queue fill levels, forwarder connections
<code>license_usage.log</code>	Bytes indexed per index, source type and host (<code>type=Usage</code>)
<code>scheduler.log</code>	Every scheduled search run: status, duration, results, actions

Source in <code>_internal</code>	Contains
<code>splunkd_access.log</code> , <code>web_access.log</code>	REST and Splunk Web requests
<code>index=_audit</code> (separate index)	Logins, searches run, configuration changes

EVERYDAY ADMIN SEARCHES

``` errors and warnings by component ```

`index=_internal sourcetype=splunkd log_level IN (ERROR, WARN) | stats count by component, log_level | sort - count`

``` licence used today per index and source type (MB) ```

`index=_internal source=*license_usage.log type=Usage earliest=@d | stats sum(b) AS b by idx, st | eval MB=round(b/1024/1024, 2)`

``` ingestion per source type over time ```

`index=_internal source=*metrics.log group=per_sourcetype_thruput | timechart span=1h sum(kb) by series`

``` blocked queues (the indexer cannot keep up) ```

`index=_internal source=*metrics.log group=queue blocked=true | stats count by name, host`

``` timestamp and line-breaking problems ```

`index=_internal sourcetype=splunkd component IN (DateParserVerbose, LineBreakingProcessor, AggregatorMinigProcessor) | stats count by component, data_sourcetype`

``` failed logins to Splunk itself ```

`index=_audit action="login attempt" info=failed | stats count by user, src`

CASE STUDY 10

The warehouse feed that never arrived

Situation. NovaMart's Mumbai warehouse vendor delivered a new stock-movement log and an `inputs.conf` for it. The admin copied the file into a new app, restarted Splunk, and told the business "it's live". A day later the supply-chain manager complains: "The dashboard shows nothing for the warehouse."

Investigation.

1. **Is there any data?** `index=novamart_inv` over All time returns nothing. Splunk Web also shows a message: "Search peer ... has the following message: Received event for unconfigured/disabled/deleted index=novamart_inv ..."
2. **What does splunkd say?** `index=_internal sourcetype=splunkd novamart_inv` confirms that events are being **dropped** because the index does not exist. Nobody created it.
3. **Preview the file.** In *Add Data* → *Upload* (without submitting), the preview shows `_time` values in **August**, although the events are from 14–20 September. Each line starts `WH-MUM batch_date=2026-08-10 ... ts=14-09-2026 06:05:57.128`: Splunk takes the first date it finds, the *batch* date.
4. **Check the inputs.** `splunk btool inputs list --debug` shows no `host` setting, so events would carry the Splunk server's host name instead of `wh-mum01`.

Fix. Create the index; add `TIME_PREFIX = ts=` and `TIME_FORMAT = %d-%m-%Y %H:%M:%S.%3N` (and `TZ`) for `novamart:inventory`; add `host = wh-mum01`. Because the file was already read (and

dropped), reset its entry in the **fishbucket** so that Splunk reads it again, then restart. Result: 630 events, 14–20 September, host `wh-mum01`, with 180 of them on the sale day.

Lessons. "Copied and restarted" is not onboarding. Always create the index first, test the source type in the Add Data preview, set the metadata explicitly, and validate the result with `tstats`. `_internal` usually explains exactly what went wrong. You will fix this feed yourself in **Lab 14**.

TRY IT

Lab 14: deploy the broken warehouse input, diagnose it with Messages, `_internal`, the Add Data preview and btool, fix it with `.conf` files, and re-index it. **Lab 15:** set index retention, create the `nm_analyst` role and a user, verify their restrictions, and tour the Monitoring Console.

KNOWLEDGE CHECK · MODULE 11

1. Why should you never edit files in a `default` directory? Where should the change go instead?
2. Which command shows the effective value of a setting and the file it came from?
3. In which file would you (a) send a new log file to an index, (b) fix a wrong timestamp, (c) drop health-check events, (d) point a forwarder at two indexers?
4. An index has `frozenTimePeriodInSecs = 7776000` and `maxTotalDataSizeMB = 1000`. It receives 50 MB of compressed data a day. Roughly how many days of data will it keep, and why?
5. How do you stop a role from seeing `linux_secure` events in the `novamart` index?
6. Which `_internal` source would you search to find (a) skipped scheduled searches, (b) licence usage per source type, (c) blocked indexing queues?

Day 2 Summary

6

MODULES

8

HANDS-ON LABS

4

NEW LOOKUPS

2

ALERTS BACKTESTED

Module	Key takeaways
6 · Statistics	<code>stats</code> for any grouping, <code>chart</code> for a matrix, <code>timechart</code> for trends. Name results with <code>AS</code> . Filter calculated results with <code>where</code> . <code>eval</code> before, inside (<code>count(eval(...))</code>) and after <code>stats</code> turns counts into KPIs such as error rates and SLOs.
7 · Lookups	File → definition → (optional) automatic lookup. Use <code>AS</code> to map field names, <code>match_type</code> for CIDR or wildcards, <code>fillnull</code> for non-matches. <code>inputlookup</code> reads, <code>outputlookup</code> writes. Enrich after <code>stats</code> when you can.
8 · Reports	Reports are saved searches plus presentation. Choose the chart from the question. Schedule with a time range that matches the interval. Share deliberately; watch <i>Run As</i> and ownership.
9 · Dashboards	Panels, inputs and tokens. Give every input a default. KPIs at the top, details below, one global time range. Use base or scheduled searches for speed, and drilldown for detail.
10 · Alerts	Scheduled beats real-time for most needs. Put the threshold in the SPL, match the window to the schedule, throttle, backtest. Debug with <code>scheduler.log</code> and <code>sendmalert</code> .
11 · Admin	Edit <code>local</code> , never <code>default</code> ; verify with <code>btool</code> . inputs/outputs/props/transforms. Retention is whichever of time or size is reached first. RBAC through roles, indexes and search filters. The Monitoring Console and <code>_internal</code> explain most problems.

Day 2 Quick Reference

STATISTICS

<code> stats count, dc(x), sum(x), avg(x), min(x), max(x) by f</code>	Aggregate per group
<code> stats count(eval(status ≥ 500)) AS errors</code>	Conditional count
<code> chart sum(amount) over region by method</code>	Matrix
<code> timechart span=1h count by host limit=5</code>	Trend
<code> bin _time span=5m stats count by _time</code>	Time buckets as rows
<code> top limit=5 f by g / rare f</code>	Rank
<code> where errors > total*0.05</code>	Filter on an expression
<code> eval pct=round(a/b*100, 2)</code>	Rate
<code> eventstats avg(x) AS avg_x</code>	Add an aggregate to each event
<code> addcoltotals / addtotals</code>	Totals row / column

LOOKUPS

<code> lookup def key AS field OUTPUT a, b</code>	Enrich
<code> inputlookup def where x="y"</code>	Read a table
<code>... outputlookup file.csv [append=true]</code>	Write a table
<code>NOT [inputlookup def fields clientip]</code>	Exclude listed values
<code>match_type = CIDR(cidr)</code>	IP-range matching

TIME

<code>strftime(_time, "%H") / "%A"</code>	Hour / weekday
<code>-5m@m → @m</code>	Alert window for */5
<code>-7d@w1 → @w1</code>	Previous Monday–Sunday
<code>0 8 * * 1</code>	Cron: Monday 08:00

DASHBOARDS AND ALERTS

<code>host=\$host\$</code>	Token in SPL
<code>\$global_time.earliest\$</code>	Studio time token
<code>\$row.status.value\$ / \$click.value\$</code>	Drilldown values
<code>\$result.errors\$, \$results_link\$</code>	Alert message tokens
<code>index=_internal sourcetype=scheduler</code>	Did it run?
<code>component=sendmodalert</code>	Did the action work?

ADMINISTRATION

<code>splunk btool <conf> list -- debug</code>	Effective config
<code>splunk restart</code>	Apply .conf changes
<code> rest /services/data/indexes</code>	Index settings
<code>frozenTimePeriodInSecs</code>	Time retention
<code>srchIndexesAllowed, srchFilter</code>	Role data access
<code>source=*license_usage.log type=Usage</code>	Licence per index

Glossary

Term	Definition
Aggregation function	A function that summarises many values into one: count, sum, avg, dc...
Alert action	What happens when an alert triggers: email, webhook, log event, lookup, script.
Automatic lookup	A lookup applied to every search of a source type, source or host, without the <code>Lookup</code> command.

Term	Definition
Backtest	Running alert logic over past data to see when and how often it would have fired.
btool	CLI tool that shows the merged, effective configuration and where each setting came from.
Capability	A permission to perform an action (e.g. <code>schedule_search</code>), granted through roles.
Chain / base search	A dashboard search whose results feed several panels (post-processing).
Drilldown	What happens when a user clicks a chart or table: open a search, set a token, link.
Fishbucket	Splunk's record of how far each monitored file has been read (reset it to re-index a file).
Frozen	Data past retention: deleted, or archived if <code>coldToFrozenDir</code> is set.
Lookup definition	A named configuration pointing to a lookup table, with matching options.
Monitoring Console	Built-in dashboards for the health and performance of Splunk itself.
Orphaned search	A scheduled search whose owner no longer exists; it stops running.
Run As (Owner/User)	Whose permissions a shared scheduled search uses.
SLO / error budget	A target for reliability (e.g. 99.5%) and the amount of failure it allows.
Throttling	Suppressing repeat alert triggers for a period, optionally per field value.
Token	A dashboard variable set by an input or a click, used as <code>\$name\$</code> in searches.

Appendix A: Knowledge Check Answers

Module 6

1. (a) `timechart span=1h avg(latency_ms) by gateway`; (b) `chart sum(amount) over region by payment_method`; (c) `stats min(_time) AS first_seen, max(_time) AS last_seen by clientip`.
2. `count` counts events; `count(error_code)` counts only events that have the field (80). `count(clientip)` counts events with the field (22,641); `dc(clientip)` counts unique values (425).
3. After `stats count by host` the only fields left are `host` and `count`; `status` no longer exists.
4. `where region="North"`. Without quotes, North is treated as a field name.
5. `index=novamart sourcetype=access_combined | stats count AS total, count(eval(status ≥ 500)) AS errors by host | eval error_pct=round(errors/total*100, 3)` → web01 43 / 10,856 = 0.396%; web02 30 / 11,785 = 0.255%.

Module 7

1. Lookup table file, lookup definition, automatic lookup. The definition gives the friendly name (you can also use the file name directly, e.g. `lookup novamart_products.csv ...`).
2. `| lookup threat_intel_ips ip AS clientip OUTPUT threat_category`
3. By default, matching is exact text; `10.20.1.15` never equals `10.20.1.0/24`. `match_type = CIDR(cidr)` enables range matching.
4. When many users and searches need the same enrichment all the time, so that the fields appear automatically.
5. `inputlookup` reads a table as results (e.g. check the contents, or build a subsearch filter); `outputlookup` writes results to a table (e.g. save a list of admin IPs, or keep a history of a scheduled report).
6. The lookup file or definition is still private (or app-only in another app). Share it with the right permissions.

Module 8

1. (a) Line or area; (b) single value (with colour ranges) or gauge; (c) stacked column.
2. `30 7 * * 1-5`
3. "Last 24 hours" at 06:00 covers 06:00 yesterday to 06:00 today, which is two partial days. `-1d@d` to `@d` covers exactly yesterday, which is right for daily business totals.
4. They become orphaned and stop running. Reassign them before deleting the user, and have service accounts own important reports.
5. The report runs with the owner's (e.g. an admin's) data access, so viewers can see data their own roles would not allow.

Module 9

1. Any two of: JSON vs XML definition; absolute and grid layouts vs rows/panels; richer visual customisation; Studio is the recommended framework, Classic is legacy but widespread.
2. A named variable substituted into searches as `$name$`. Input: the host dropdown sets `$host$`. Drilldown: clicking a status sets `$sel_status$`.
3. Without a default, panels that use the token wait for input and show nothing when the dashboard opens.
4. Use one base (chain) search for the shared part, and post-process it in each panel, or back heavy panels with scheduled reports.

5. `>` is a special character in XML. Escape it as `>` or wrap the query in CDATA.

Module 10

1. Only when every single event matters within seconds, e.g. an Internet root login. Real-time searches use search-head and indexer resources continuously.
2. The windows overlap, so each event is evaluated three times, causing duplicate triggers. Use `-5m@m` to `@m` to match the 5-minute schedule.
3. *Once* sends one action per run; *For each result* sends one per result row. Brute force: *for each result* with throttling on `src_ip`, so that each attacker is reported once and a new attacker is not hidden.
4. `$result.errors$`
5. (1) `sourcetype=scheduler`: did it run, how many results, was it suppressed or skipped? (2) `component=sendmodalert`: did the action execute? (3) `python.log` (sendemail) or the action's own log for delivery errors.

Module 11

1. Upgrades overwrite `default`. Put the change in the matching `local` directory.
2. `splunk btool <conf> list --debug`
3. (a) `inputs.conf`; (b) `props.conf`; (c) `props.conf` + `transforms.conf` (nullQueue); (d) `outputs.conf`.
4. About 20 days. The 1,000 MB size limit (1,000 ÷ 50) is reached long before the 90-day time limit; whichever comes first wins.
5. Give the role a search filter `srchFilter = NOT sourcetype=linux_secure` (or move that data to a separate index the role cannot search).
6. (a) `scheduler.log ( status=skipped )`; (b) `license_usage.log ( type=Usage , field st )`; (c) `metrics.log ( group=queue blocked=true )`.